

Code Quality & Anti-Slop Standards — Executive Summary

Early Access · MessageFoundry v0.3.2 · July 2026

Is this code good, or is it slop? · MEFOR verdict: A- · July 14, 2026

How to judge whether code — human- or AI-authored — is actually good, using signals the evidence supports.

At a glance	
MEFOR verdict	A- / low slop-risk — all 11 signals Built
Core thesis	No single number certifies quality — judge enforced structure and verified behavior
Hard rule	Never gate on coverage %, LOC, complexity, or SonarQube counts alone — all weak or gameable
Proven value	Caught private security docs shipping in the public PyPI package (fixed, verified at v0.3.0)
Evidence base	Peer-reviewed and adversarially verified — 20 claims confirmed, 5 popular claims refuted

AI slop is real — and measurable

- AI-assisted developers wrote *less* secure code while *more* confident it was secure (Stanford).
- ~14% of distinct Python modules in real ChatGPT output didn't exist ("slopsquatting").
- 2024: copied lines exceeded refactored lines for the first time in industry telemetry.

Each failure mode maps to a specific machine-enforced control — not a vibe check.

What actually predicts quality

Machine-enforced structure (layer boundaries, strict typing), tests whose assertions are validated by mutation testing, and dependency + published-artifact integrity. The popular scoreboards — coverage %, complexity scores, SonarQube severities — correlate weakly, sometimes *invertedly*, with real defects.

The MEFOR result

All 11 signals Built and running in CI: enforced boundaries, strict typing, 5,400+ behavior-verifying tests, locked dependencies, 11 security scanners. The one durable gap ever found — the PyPI leak — was caught by this rubric and closed.

The 11 Signals at a Glance

Simplified from the full rubric (§4) and MEFOR scorecard (Appendix A), which follow. A codebase is judged by the **composite** — never by any single row.

Tier 1 — durable, high-signal controls (these carry the verdict)

#	Signal	What it asks	MEFOR
1	Architecture boundaries	Are module/layer rules machine-checked in CI, not just documented?	✓ Strong
2	Strict typing	Full strict type-checking, no blanket suppressions?	✓ Strong
3	Tests verify behavior	Do tests assert real values and failure paths — not mock choreography?	✓ Strong
4	Dependency integrity	Is every dependency verified to exist and hash-locked?	✓ Strong
5	Security scanning + threat model	Blocking scanners plus a written threat model and review?	✓ Strong
6	Published-artifact integrity	Do released packages ship <i>only</i> intended content?	✓ Built — caught and fixed a real leak

Tier 2 — measurement layer (guidance and triage — never a gate on its own)

#	Signal	What it asks	MEFOR
7	Mutation testing	Do the tests actually catch injected bugs?	✓ Built (advisory)
8	Coverage visibility	Is coverage reported on changed lines, as guidance?	✓ Built (advisory)
9	Duplication / reuse	Is new copy-paste flagged for review?	✓ Built (advisory)
10	Lint breadth	Is a broad static-analysis ruleset enforced?	✓ Built — enforced in the required CI leg
11	Complexity triage	Are the genuinely large units surfaced for review?	✓ Built (advisory)

The anti-metric rule: none of the Tier 2 numbers may ever be *the* quality gate. They surface problems for a human to judge; the Tier 1 structure carries the verdict.

The full standard follows: the evidence review, the AI failure-mode map, the complete rubric, gate placement, the MEFOR scorecard, and the cited methodology.

Code Quality & Anti-Slop Standards — Evidence-Based Rubric for Judging Code (Human- or AI-Authored)

A companion standard to *Secure Development Standards (SDS)* and *Secure AI-Assisted Development Standards (the AI-build companion)*. The SDS says what a secure build must satisfy; the AI-build companion says how to build it with an AI assistant (process, tiers, provenance). This document is the third leg: how to judge whether the resulting code is actually good — not "AI slop" — using signals the evidence supports, not scoreboards it refutes. It governs the outcome, where the SDS governs the process and the AI companion governs the tooling.

Scope boundary — read this first. This is a **code-quality measurement rubric**, not a security standard (that is the SDS) and not the AI-build process standard (that is the **Secure AI-Assisted Development Standards**). Where a signal here is *already owned* by a companion — dependency-existence verification, human review, SAST — this doc **points to it and does not restate it**. Its own additions are the *quality-measurement* gates neither companion carries: test-signal proof (mutation), duplication/reuse detection, coverage *visibility*, complexity *triage*, lint breadth, and **published-artifact / supply-chain-out integrity** (signal 6).

Document	Code Quality & Anti-Slop Standards
Applies to	Any project developed under the SDS. MessageFoundry (MEFOR) is the reference implementation (Appendix A); future projects add Appendix B, C, ...
Maintained by	Project maintainers (open-source). Each deploying/adopting organization assigns its own local owner.
Status	Draft for review
Version	0.11
Date	July 27, 2026
License	Publishable under the project's open-source license; intended to be shared with adopters and reused across projects.
Review cadence	At least annually, and on any material change to the metric evidence base or the AI toolchain.
Aligns to	ISO/IEC 25010:2023 (product-quality model — Maintainability = modularity / reusability / analyzability / modifiability / testability) · companion to SDS PW.7 / PW.8 and the Secure AI-Assisted Development Standards §3 failure-modes / §9 deferred-gates. Evidence base is peer-reviewed metric-validity studies + DORA 2024 + GitClear + the METR RCT + Stanford CCS'23 , each carried with its honesty caveat (§7). Confers no certification.

Executive summary

The core thesis, and it is counterintuitive: *there is no single number that certifies code quality, and every metric people reach for first is a weak or gameable predictor.* Peer-reviewed evidence shows line-coverage %, raw cyclomatic complexity, SonarSource "Cognitive Complexity", and SonarQube issue severities all correlate **weakly — sometimes invertedly** — with real defects and change-proneness (§2). What survives is **structure and behavior, verified by enforced controls**: ISO/IEC 25010 maintainability (low coupling / information hiding), strict typing, tests whose *assertions* are validated (mutation testing as *guidance*, not a gate), dependency integrity, and static analysis in the loop.

"AI slop" is real but specific. The evidence names measurable failure modes — a controlled Stanford study found AI-assisted developers wrote *less* secure code yet were *more* confident it was secure; GitClear telemetry shows 2024 was the first year copy/pasted lines exceeded refactored ("moved") lines (the signature of *copy-instead-of-abstract*); ~14.4% of *distinct* Python modules in real ChatGPT output were hallucinated (slopsquatting); DORA 2024 associated AI adoption with a delivery-**stability** drop. The answer to each is a **specific control**, not a vibe (§3).

This document is a measurement rubric, not another process gate. It gives (1) the signals that genuinely separate good code from slop, with the honest note that **no validated single-metric cutoff exists** — thresholds are set empirically per project; (2) an anti-metric list of what **not** to gate on; and (3) a scored MEFOR scorecard (Appendix A). **MEFOR's verdict: strong exactly where the evidence says it counts (machine-enforced structure, strict typing, behavior-verifying tests, dependency integrity, security scanning), and — as of this cycle — the *measurement* layer has closed too: mutation, coverage visibility, clone detection, complexity triage, and the broadened lint ruleset all ship as CI gates (#1028/#1040/#1047), so every one of the 11 signals is now Built.** The one *durable-control* gap this cycle surfaced — a private-doc leak in the published PyPI sdist — was caught via signal 6 and fixed (#1020), verified clean at v0.3.0.

1. Purpose, scope, and the lens

This rubric answers one question: **"Is this code good, or is it slop?"** — for code that may be human- or AI-authored, in a repository built largely with an AI assistant across many parallel sessions.

It serves three audiences: **maintainers** (a standing scorecard to re-run each release), **adopters and auditors** (evidence the code is judged against the evidence, not a badge), and **future projects** (their own Appendix).

The lens — structure over scoreboards. The evidence is unambiguous that *single-number gates fail*. Therefore this rubric is **composite and structural**: it weights machine-enforced architectural boundaries and validated test signal far above any coverage or complexity number, and it explicitly **forbids** certifying quality on a single metric (§4). This is the quality-measurement analogue of the SDS's "deterministic checks, never ask the model to be secure" principle — *measure structure and behavior, never trust a scoreboard*.

2. The evidence — validated vs. gameable (read before setting any gate)

Peer-reviewed, **adversarially-verified** findings on what actually predicts quality (full citations → [Appendix B.4](#)):

Popular metric	Verdict	Evidence
Cyclomatic complexity (raw)	Weak predictor; largely a proxy for LOC. Use as a local <i>triage smell</i> , never a gate.	Correlation with real bugs ≈ 0.06 (Kendall); "adds little if any" beyond executable-line counts. [R2]
SonarSource "Cognitive Complexity"	No incremental predictive value over traditional measures.	Peer-reviewed JSS evaluation: "does not appear to fulfill the promise." [R3]

Popular metric	Verdict	Evidence
SonarQube quality-gate severities	Weak, inconsistent, sometimes <i>inverted</i>. Flagged "dirty" classes are no more fault-prone than clean ones. Useful as a cheap filter, not a quality score.	33 Apache projects, ~27K faults. [R4]
Mutation score (as a single number)	Poor <i>linear proxy</i> (mostly a test-suite-size artifact) — but mutation testing is high-value as <i>guidance</i> : top-decile suites catch 8–46% more real faults.	ICSE 2018 (Papadakis et al.). [R5]
Line-coverage % / LOC	Gameable. High coverage with weak assertions is the canonical AI-slop hiding place; LOC measures size, not quality.	Corollary of the mutation and complexity findings above.

What survives as durable signal: ISO/IEC 25010:2023 [R1] maintainability — decomposed into *modularity, reusability, analyzability, modifiability, testability*, i.e. **low coupling + information hiding** — enforced as **architectural fitness functions** (import/layer boundaries), plus **strict typing, behavior-verifying tests validated by mutation testing, dependency integrity, and static analysis in the loop.**

Honesty note on the hype. Several widely-quoted "AI slop" statistics **failed adversarial verification** during this document's research: the "~40% of Copilot code is vulnerable" figure, a "10× duplicate-block surge", and two GitClear copy/move percentages were all refuted — full list in [Appendix B.5](#). **Judge on the structural signals below, not on alarm-bell numbers.** (Full caveats: §7.)

3. AI-specific slop failure modes → the control that neutralizes each

The [Secure AI-Assisted Development Standards §3](#) names *five process failure modes* (intent drift, context rot, error accumulation, the speed–quality paradox, misplaced trust). This rubric maps the **code–outcome** failure modes those produce to a measurable control:

AI slop failure mode	Evidence	The control (owner)
Insecure code + overconfidence	Stanford CCS'23: AI-assisted users wrote less-secure code, <i>more</i> confident it was secure. [R6]	Mandatory human review + blocking SAST that cannot be waived — owned by SDS PW.7 + AI companion §6.5/§6.6 . This rubric only <i>checks it is present</i> .
Hallucinated / typosquatted dependencies ("slopsquatting")	~14.4% of <i>distinct</i> Python modules in real ChatGPT output did not exist. [R7]	Verify-before-add + hash-locked lockfile + new-import audit — owned by AI companion §6.4/§9 . This rubric <i>checks it is present</i> .

AI slop failure mode	Evidence	The control (owner)
Silent duplication over reuse (copy-instead-of-abstract)	GitClear: 2024 was the first year copy/pasted lines (12.3%) exceeded "moved"/refactored lines (9.5%). [R9] (<i>Correlational</i> — §7.)	Clone-detection on the diff + a "moved vs copied" review lens. New gate — this document (§5).
Shallow tests that assert little	Coverage % hides assertion-free tests; mutation testing exposes them. [R5]	Mutation testing on changed code, as guidance. New gate — this document (§5).
Unbounded complexity / over-abstraction	Large, tangled units are a maintainability smell (though a weak <i>defect</i> predictor — §2).	Advisory complexity triage (surface, don't gate). New gate — this document (§5).
Inconsistent conventions across a codebase	Multi-session AI authorship drifts style/structure.	Broad lint ruleset + <code>CLAUDE.md</code> as the standing convention anchor. Partly new (lint breadth) — §5.
Velocity ≠ delivered quality	DORA 2024: AI adoption associated with ~7.2% lower delivery <i>stability</i> per 25% adoption. [R8] · [R10] (<i>Correlational, partly revised in 2025</i> — §7.)	Small batch sizes + DORA change-fail/MTTR awareness. Owned by AI companion §3.
Control-parity gap (a guard on one path, missed on its sibling)	AI companion §6.6: an AI implements a control <i>where prompted</i> and misses its siblings — every confirmed medium-or-higher finding in the 2026-06 MEFOR audit had this shape. Real instance: the fail-closed leak gate guarded the <i>git-mirror</i> publish path but not its sibling, the <i>PyPI</i> publish path, so the sdists shipped private docs on every release.	Enumerate sibling paths for every control; encode as one deterministic check where feasible. Here → the published-artifact-integrity gate (signal 6).

4. The rubric — 11 signals that separate good code from slop

Each signal is a **risk** → **control** → **measure**, tagged by **gate type** (deterministic = machine-checked; advisory = human arbitrates) and by which document **owns** it. The signals fall into **two tiers** — weight them accordingly. **A codebase is certified "not slop" by the composite, never by any single row (§2).** Signals are numbered **contiguously by tier: Tier 1 = 1–6, Tier 2 = 7–11.**

Tier 1 — Durable, high-signal controls (weight these most). Structural, machine-enforceable properties where quality is *hard to fake* — the ones the evidence says actually predict maintainability and catch slop. These carry the verdict.

#	Signal	What "good" looks like	Gate type	Owner
1	Enforced architecture boundaries (ISO 25010 modularity / low coupling)	Import/layer rules are <i>machine-checked in CI</i> , not just documented	Deterministic	SDS PW.1–2; checked here

#	Signal	What "good" looks like	Gate type	Owner
2	Strict typing	<code>mypy --strict</code> ; suppressions carry error codes (no blanket ignores)	Deterministic	AI companion §6.5; checked here
3	Tests verify behavior, not mocks	Value/negative-path assertions; real integrations over mock choreography	Deterministic	SDS PW.8; checked here
4	Dependency integrity (anti-slopsquatting)	Existence-verify + hash-locked lockfile + new-import audit	Deterministic	AI companion §6.4/§9 (pointer only)
5	Security scanning + threat model	Blocking SAST/SCA/secret-scan + written threat model + human review	Deterministic + advisory	SDS PW.7 / AI companion §6.5–6.6 (pointer only)
6	Published-artifact integrity (supply-chain-out)	Published sdist/wheel ship only intended content — the package manifest is an <i>allowlist</i> (not a whole-repo sweep) + a fail-closed publish gate blocks any private/unintended file before the irreversible upload. Distinct from row 4's <i>incoming</i> dependency integrity	Deterministic	This document — new (+ SDS supply-chain)

Tier 2 — Measurement / lower-signal layer (guidance & triage — never a gate on their own). Useful for *surfacing* problems, but each is a weak or gameable predictor in isolation (§2, §4.1), so they inform review — they do not certify quality.

#	Signal	What "good" looks like	Gate type	Owner
7	Test-signal proof (mutation)	Mutation testing on changed code exposes shallow tests	Advisory (guidance)	This document — new
8	Coverage visibility	Coverage <i>measured on the diff</i> as guidance — never a whole-repo % gate	Advisory	This document — new
9	Duplication / reuse discipline	Clone-detection flags <i>new</i> duplication in diffs; justified parity whitelisted	Advisory	This document — new
10	Lint breadth (static analysis in the loop)	Broad ruleset (bugbear, comprehensions, simplify, pyupgrade, isort)	Deterministic	This document — expand existing
11	Complexity as triage (never a gate)	Advisory complexity signal surfaces the genuinely-large units	Advisory	This document — new

Context caveat, not a signal — delivery stability (DORA). "Velocity $\neq$ delivered quality" is a real AI-slop concern (DORA 2024 associated AI adoption with lower delivery *stability*), but it measures **delivery outcomes**, not whether a given diff is slop — a different altitude from the code-artifact signals above, on weaker (correlational, partly-revised — §7) evidence, and **owned by the AI companion §3**. MEFOR's small-batch discipline (one coherent layer per commit) already covers the actionable part. It stays a §3 failure-mode entry, **not** a peer signal here.

Evidence & citations for the matrix. Every signal and claim above maps to its supporting study in **Appendix B.3** (per-element evidence table), with full bibliographic citations in **Appendix B.4**, the derivation method in **Appendix B.2**, and the claims that *failed* verification in **Appendix B.5**.

4.0 The liveness rule (hard) — a gate that cannot fail is not a control

Every advisory gate must prove it measured something, or say why it could not. A gate that reports a conclusion without recording that it performed a measurement is indistinguishable from one that worked, and it will stay that way indefinitely, because nobody investigates a green check.

This rule was written from failure, not theory. Three defects across **two** of this document's own Tier 2 gates were found (2026-07-27) to have been green throughout: two gates measuring nothing (one of them for two full versions of this rubric, during which it was scored  **Built** here — v0.10), and one gate that measured correctly but published a wrong derived number. Section 4.1 protects against trusting a *number* too much; nothing protected against trusting a *green check that never ran*. That is a distinct failure mode and it needs its own control:

1. **Proof of execution, not of findings.** The receipt counts units *examined* — files scanned, mutants processed, changed lines analysed — never units *found*. A clean codebase legitimately reports zero clones, and a liveness check that fires on good news gets muted, leaving the project worse off than before it existed.
2. **"Nothing to measure" is acceptable only when stated.** An explicit, reasoned declaration passes; a silent empty result fails. The two are visually identical, which is precisely why the reason is load-bearing.
3. **Reported numbers must reconcile against an INDEPENDENT source.** A gate can execute perfectly and still emit a wrong derived figure. Two rules, because the obvious one is weaker than it looks: parts must sum to the whole, each counted independently and never as a remainder (a remainder makes the sum true by construction); *and* any derived headline figure must be cross-checked against a second, independently produced measurement of the same quantity. A sum that includes a derived term can be algebraically blind to that term — ours was, and the blindness is now asserted by a test rather than assumed away.

Applies to any gate, in any project adopting this rubric — a deferred or advisory gate that silently stops measuring is worse than an absent one, because the scorecard still counts it.

4.1 The anti-metric rule (hard)

Do NOT certify quality — or fail a build — on any single one of: line-coverage %, LOC, raw or cognitive cyclomatic complexity, or SonarQube severity counts. Each is a weak or gameable predictor (§2). They may be *surfaced as advisory triage signals*; they must never be *the* quality gate. This mirrors the AI companion's "gates are deterministic checks, never ask the model to be secure" — here: *a scoreboard is never the verdict*.

4.2 No validated single threshold (honest)

The literature validates that single metrics fail, but supplies **no validated numeric cutoff** for a combined scorecard. Thresholds (e.g. a mutation-score floor on changed code, a max new-clone count) are therefore **set empirically per project and recorded in the Appendix**, reviewed as data accumulates — not imported as universal constants. Where this document names a directional target, it is flagged as project-set, not evidence-certified.

5. The new measurement gates — placement (local vs CI) and status

The five gates this document adds (rubric rows 7–11) are *quality-measurement* gates. Placement follows the project's **three enforcement points** (identical to the AI companion §6.5): **pre-commit hook** (local), **messagefoundry check** (local + CI + IDE), and **GitHub Actions CI** (the authoritative gate, incl. self-hosted Windows runners).

Gate	Cost	Recommended placement	Blocking?	Taxonomy
Advisory complexity triage (<code>C901</code> via <code>ruff</code>)	Cheap	CI advisory job	Advisory only (never a hard gate — §4.1)	✔ Built — <code>quality-advisory.yml</code> (#1028)
Clone-detection (<code>jscpd</code>)	Cheap	CI advisory job; store-parity whitelisted	Advisory (finding, not fail)	✔ Built — <code>quality-advisory.yml</code> (#1028)
Diff-coverage visibility (<code>diff-cover</code>)	Moderate	CI leg (reports on changed lines)	Advisory	✔ Built — <code>quality-advisory.yml</code> (#1040; PR-only)
Mutation testing (<code>mutmut</code> , bounded scope)	Cheap — measured (461 mutants in 3s; <code>mutmut</code> 3 runs only the tests covering each mutant)	CI leg (advisory), including PRs + opt-in local command	Advisory (guidance)	✔ Built — <code>quality-advisory.yml</code> (#1040; repaired 2026-07-27, see v0.10)
Expand ruff ruleset (<code>B</code> , <code>C4</code> , <code>SIM</code> , <code>UP</code> , <code>I</code>)	Cheap	The required CI <code>ruff check</code> leg	Blocking (from a clean baseline)	✔ Built — <code>pyproject.toml</code> <code>extend-select</code> (#1047)

Answering "are these local?" — Cheap gates (`ruff` breadth, `C901`) run **both** locally (pre-commit) and in CI. The expensive gates (mutation, clone, diff-coverage) are **CI-first** — they are too slow for the inner loop — but each is **exposable as an opt-in local command** so a maintainer can run it on demand. The *authoritative* enforcement is CI in every case; local runs are for fast feedback. This split is deliberate: it keeps the local edit-loop fast while the diff-scoped heavy analysis runs where wall-clock doesn't block the human.

All five measurement gates are now Built. Complexity (11) and clone (9) shipped first (#1028), then diff-coverage (8) and mutation (7) (#1040) — non-required advisory jobs in `quality-advisory.yml` that can never block a merge — and finally the **ruff-breadth expansion (signal 10, #1047)**, which unlike the others is enforced by the required `ruff check` leg (from a grandfathered clean baseline; new code must comply). Nothing remains designed-but-deferred.

6. How this maps to the companion standards

This rubric	Companion anchor
Rows 1–2 (structure, typing)	SDS PW.1–PW.2 (secure design, modularity); AI companion \$6.5 (verification gates)
Row 3 (behavior-verifying tests)	SDS PW.8 (test executable code) — <i>this rubric adds the quality bar on top of presence</i>
Row 4 (dependency integrity)	AI companion \$6.4 / \$9 — <i>owned there; checked here</i>
Row 5 (security review + SAST)	SDS PW.7 ; AI companion \$6.5–6.6 — <i>owned there; checked here</i>
Row 6 (published-artifact integrity)	New — this document (+ SDS supply-chain)
Rows 7–11 (measurement gates)	New — 7/8/9/11 Built (advisory) in <code>quality-advisory.yml</code> ; 10 Built + enforced by the required <code>ruff check</code> leg (#1047)
Delivery stability (DORA) — <i>context caveat, not a signal</i> (§4)	AI companion \$3 (the METR / DORA calibration)
The anti-metric rule (§4.1)	The quality-side analogue of AI companion \$5 "gates are deterministic checks, never ask the model to be secure"

7. Evidence caveats (carry these into every use)

1. **Metric-invalidation findings are robust** (peer-reviewed primary studies) but are "*this metric is weak alone*" results — they argue against **single-number gates**, not against measurement.
2. **The GitClear duplication/churn trend is descriptively solid but the AI causation is correlational**, from a commercial vendor using a proprietary "moved" reuse heuristic, and confounded (2022–24 layoffs, a startup-growth effect). Treat the duplication *trend* as real; treat the AI attribution as interpretation.
3. **Several AI-code findings are model-era-specific.** The Stanford security study used codex-davinci-002 (2022); the package-hallucination figures skew to GPT-3.5-era output. **Frontier models plausibly perform better — re-baseline periodically.**
4. **The 14.4% Python hallucination figure is share of *distinct* modules**, not per-import rate (which is far lower — common modules are never hallucinated).

5. **DORA 2024's negative delivery effect is associational** and was partly revised in DORA 2025.
6. **No source provides a validated single-metric certification threshold** — the evidence supports composite, structural, guidance-based assessment (§4.2).

8. References

Quick links below. Full bibliographic citations, the per-element evidence map, and the derivation method are in *Appendix B* (B.4 / B.3 / B.2).

- **ISO/IEC 25010:2023** — Systems and software Quality Requirements and Evaluation (product-quality model).
- **Cyclomatic complexity vs. defects** — [arXiv 1912.01142](#).
- **Cognitive Complexity has no incremental predictive value** — [JSS 2022](#).
- **SonarQube technical-debt items weakly/invertedly related to faults** — [arXiv 1908.11590](#).
- **Mutation score is a size artifact but valuable as guidance** — [ICSE 2018](#), [Papadakis et al.](#)
- **AI assistants → less-secure, more-confident code** — [Stanford, CCS 2023](#), [arXiv 2211.03622](#).
- **Package hallucination / slopsquatting (~14.4% distinct Python modules)** — [WildCode](#), [arXiv 2512.04259](#).
- **DORA 2024** — AI adoption vs. delivery stability/throughput — [dora.dev/research/2024](#).
- **GitClear** — copy/paste vs. moved lines, churn — [GitClear AI Copilot Code Quality 2025](#).
- **METR RCT (experienced devs 19% slower)** — cited via the [AI companion §3/§11](#).
- **Cross-links:** [Secure Development Standards](#) · [Secure AI-Assisted Development Standards](#) · [../CLAUDE.md](#).

Appendix A — Scorecard: MessageFoundry (MEFOR)

Scored from a read-only repo audit (2026-07-13). Future projects add Appendix B, C, ... with identical headings. Each signal is tagged *Built* / *designed-but-deferred* / *aspirational*, per the house honesty taxonomy.

A.1 Verdict

A– / low slop-risk. MEFOR implements **all six durable, high-signal controls** (rubric rows 1–6) as **Built**, and its *measurement* layer has now closed as well: **complexity (11)** and **clone (9)** shipped as advisory gates (#1028), then **mutation (7)** and **diff-coverage (8)** (#1040), and finally the **ruff-breadth expansion (#10)** (#1047, enforced by the required `ruff check` leg). So **all 11 signals are now Built**. It is strong where faking is hardest (machine-enforced structure) and thin only where the metrics are gameable anyway.

The rubric earned its keep this cycle. Applying **signal 6** (published-artifact integrity) surfaced a real **control-parity** gap (§3): the PyPI **sdist** was shipping the private security-posture docs on *every* release, because the fail-closed leak gate covered the git-mirror publish path but not its sibling, the PyPI path. It was fixed (#1020: a `[tool.hatch.build.targets.sdist]` allowlist + a fail-closed "sdist is package-only" gate in `release.yml`) and **verified clean at v0.3.0**. That found-and-fixed leak is the one *durable*-control gap that has now closed; the rest of the gaps are all in the measurement layer.

A.2 Scored signals (from the audit)

Tier 1 — durable, high-signal controls (signals 1–6): all Built .

#	Signal	Status	Evidence in repo
1	Enforced architecture boundaries	 Built — Strong	<code>tests/test_dependency_boundaries.py</code> AST-scans engine packages, blocks <code>fastapi / pyside6 / api / console</code> imports, in the required CI <code>test</code> leg
2	Strict typing	 Built — Strong	<code>[tool.mypy] strict = true</code> , dual-platform CI; all 33 <code># type: ignore + 100 # noqa</code> carry rule codes; no blanket ignores
3	Tests verify behavior, not mocks	 Built — Strong	5,402 test functions; ~7,600 value- <code>==</code> asserts; ~1,000 <code>pytest.raises</code> ; 0 <code>assert_called*</code> ; live SQL Server + Postgres integration legs
4	Dependency integrity	 Built — Strong	Hash-locked <code>requirements.lock</code> (DEP-1 lock-sync + <code>--require-hashes</code> CI); pip-audit; <code>CLAUDE.md</code> /AI-companion verify-before-add rule
5	Security scanning + threat model	 Built — Strong (<i>caveat A.4</i>)	11 scanners (CodeQL, semgrep, bandit, gitleaks, pip-audit, crypto-inventory, forbidden-content, Trivy, Scorecard, zizmor, npm-audit); SECURITY.md (735 ln) + PHI.md (688 ln)
6	Published-artifact integrity (supply-chain-out)	 Built — found & fixed this cycle	Was  : the PyPI sdist swept the whole repo, shipping <code>docs/security/*</code> , <code>CLAUDE.md</code> , <code>scripts/publish/*</code> on releases 0.1.0..0.2.15 (the mirror leak-gate never covered the PyPI path — a control-parity miss, §3). Fixed #1020: <code>[tool.hatch.build.targets.sdist] only-include</code> + a fail-closed "sdist is package-only" gate in <code>release.yml</code> ; v0.3.0 verified package-only against the live PyPI artifact (sha256 download). Historical 0.1.0..0.2.15 sdists remain public (owner-only PyPI deletion).

Tier 2 — measurement / lower-signal layer (signals 7–11): all 5 Built (#7, #8, #9, #11 advisory; #10 enforced).

#	Signal	Status	Evidence in repo
7	Test-signal proof (mutation)	 Built (advisory) — #1040, repaired 2026-07-27 (v0.10)	<code>quality-advisory.yml</code> runs <code>mutmut==3.6.0</code> over one bounded, well-tested pure module (<code>parsing/binary.py</code> ↔ <code>test_binary_carriage.py</code>), on PRs + nightly cron + <code>workflow_dispatch</code> . Measured: 461 mutants in 3s — 87 killed, 19 survived, 355 not covered by the scoped test. Survivor table in the step summary. <i>Was scored Built on <code>mutmut<3</code> from 0.8 to 0.9 while producing nothing — see v0.10.</i>
8	Coverage visibility	 Built (advisory) — #1040, surfaced 2026-07-27 (v0.10)	<code>quality-advisory.yml</code> runs <code>pytest-cov</code> + <code>diff-cover</code> on the PR's changed lines (<code>--fail-under=0</code>), PR-only — coverage <i>of the diff</i> , never a whole-repo % gate (§4.1). Now emits inline <code>::notice</code> annotations on the Files changed tab (<code>--format github-annotations:notice</code>), adjacent uncovered lines coalesced into ranges. Advisory.

#	Signal	Status	Evidence in repo
9	Duplication / reuse detection	✅ Built (advisory) — #1028	<code>quality-advisory.yml</code> runs <code>jscpd</code> on <code>messagefoundry/</code> , whitelisting the ~21k-LOC justified store-backend parity (<code>sqlserver.py</code> / <code>postgres.py</code>); surfaces <i>unjustified</i> copy-paste for triage, non-blocking
10	Lint breadth	✅ Built — #1047	<code>[tool.ruff.lint] extend-select = ["B","C4","SIM","UP","I"]</code> , enforced by the required <code>ruff check</code> leg. B008 (FastAPI DI, ~460 hits) handled via <code>extend-immutable-calls</code> + a route-layer per-file ignore (real <code>x=list()</code> bugs still caught); 515 violations auto-fixed (import sort, pyupgrade, safe simplify); 235 non-auto-fixable grandfathered with per-line <code># noqa</code> → clean baseline, new code must comply
11	Complexity triage	✅ Built (advisory) — #1028, sharpened 2026-07-27 (v0.10)	<code>quality-advisory.yml</code> runs <code>ruff --select C901 --exit-zero</code> (advisory, never gates), plus a merge-base-vs-HEAD delta (<code>scripts/quality/c901_delta.py</code>) that reports only functions a PR <i>introduced or made worse</i> . Re-measured 2026-07-27: 122 functions exceed C901 > 10 across 43 files (was 85 on 2026-07-13), complexity 11 / 14 median / 320 max. The raw list is unusable as a diff signal — all 122 findings anchor on a single <code>def</code> line — which is what the delta exists to fix

A.3 The gaps, ranked → buildable gates

Ordered by anti-slop leverage, not effort (build placement per \$5). ✅ = **shipped** (advisory; #1028 or #1040):

- Mutation testing** — highest leverage; directly counters shallow-test slop, extra weight under the solo-maintainer review deviation (A.4). ✅ **shipped** (#1040 — `mutmut` over a bounded, well-tested module; runs on PRs + nightly cron + `workflow_dispatch`; widen the scope later). (*v0.11: was "mirror-nightly" — there is no mirror post-cutover, and the job no longer carries a repo-slug gate.*)
- Clone-detection on diffs** — ✅ **shipped** (`jscpd`, store-parity whitelisted) — catches the copy-instead-of-abstract signature the parallel-worktree workflow is most exposed to.
- Diff-coverage visibility** — measured on changed lines, guidance only (never a whole-repo % gate — \$4.1). ✅ **shipped** (#1040 — `pytest-cov` + `diff-cover`, PR-only).
- Advisory C901 complexity** — ✅ **shipped** (advisory triage).
- Expand ruff select** (`B`, `C4`, `SIM`, `UP`, `I`) — ✅ **shipped** (#1047 — extend-select enforced by the required `ruff check` leg; 515 auto-fixed, 235 grandfathered from a clean baseline).

All gates are now shipped. The ruff sweep (#10, #1047) was run in a quiescent-worktree window (a 100+-file import sort would collide with in-flight parallel sessions) after pruning the stale worktrees to a minimal set. Mutation and diff-coverage were built *blind via CI* — verified by their own gate runs, since this repo's sessions can't stand up a local venv (see #1040).

Rollout record (measured 2026-07-13 — how the #1047 sweep was executed): `B,C4,SIM,UP,I` = **853 violations** (238 `B008` FastAPI false positives to exclude; 111 `I001` repo-wide import reorder); `C901` = **85 hits**. Safe rollout: (a) exclude framework-idiom rules (`B008` on `api/`); (b) **grandfather** the existing backlog so the *required* gate stays green (per-file-ignores / ratchet — new code only); (c) run the repo-wide import sort as a

dedicated pass when parallel worktrees are quiescent — a 100+ -file sweep conflicts with in-flight sessions; (d) keep **C901 advisory**. (The built coverage/mutation gates install their tools CI-side via an ephemeral **uv pip install**, so they needed **no requirements.lock** change — DEP-1 unaffected.)

A.4 Documented caveat — solo-maintainer review

Row 5's "human review" is **self-review** (the SDS §A.6 / **AI companion Appendix A.6** single-maintainer deviation). The Stanford overconfidence finding (§3) bites hardest exactly when the author reviews their own AI-authored code — which is the strongest argument for the mutation gate (Built this cycle — #1040), since it is the one control that *adversarially* checks whether the tests assert anything, independent of the author's confidence.

Appendix B — The rubric matrix: methodology & cited references

This appendix explains how to read the §4 signal matrix, how it was derived, and the evidence behind each element — with full citations. The §2, §3, and §4 tables link here.

B.1 What "the matrix" is and how to read it

The rubric's core is the **§4 matrix**: 11 signals, each a **risk** → **control** → **measure**, tagged by **gate type** (deterministic = machine-checked; advisory = human arbitrates) and by which document **owns** it. Read it as a **composite**, not a checklist of independent boxes:

- **Rows 1–6 are durable, high-signal controls** — enforced structure (ISO/IEC 25010 modularity), strict typing, behavior-verifying tests, dependency integrity, security scanning, and published-artifact integrity. These are where quality is *hard to fake*.
- **Rows 7–11 are the measurement / lower-signal layer** — mutation, coverage, clone-detection, lint breadth, complexity — useful as *guidance / triage*, never as a single gate.
- **The anti-metric rule (§4.1)** forbids certifying quality on any one number (coverage %, LOC, raw or cognitive complexity, SonarQube severity), because the evidence shows each is a weak or gameable predictor (B.3).
- **Delivery stability (DORA) is deliberately not a signal** — it measures delivery outcomes, not the code artifact, on weaker evidence; it is kept as a context caveat under the §4 table.

A codebase is judged "not slop" by the **composite** of the durable controls plus the guidance signals — with thresholds set **empirically per project** (§4.2), because no source validates a universal single-metric cutoff.

B.2 How the matrix was derived

The matrix is **evidence-informed and adversarially verified**, not authored from opinion:

1. **Adversarially-verified deep-research pass**. The question was decomposed into **five search angles** — (a) academic metric-validity, (b) quality frameworks & delivery metrics, (c) AI-slop empirical trends, (d) security & correctness studies, (e) practitioner controls. Parallel searches fanned out → ~24 sources fetched → ~109 candidate claims → the load-bearing ones put through **3-vote adversarial verification** (each verifier tried to *refute* the claim; ≥2 refutations killed it). Result: **20 confirmed, 5 refuted** (B.5). Only survivors entered the rubric.

2. **Structural scaffold — ISO/IEC 25010:2023 [R1]**: the international product-quality model, whose *maintainability* characteristic (modularity, reusability, analyzability, modifiability, testability) supplies the "structure over scoreboards" backbone (signals 1–6).
3. **The MEFOR scorecard (Appendix A)** is a separate, read-only audit of the actual repository — evidence, not estimates.
4. **Honesty discipline (§7)** — every claim carries its limitation; correlational, vendor-sourced, and model-era-specific evidence is labelled as such.

B.3 Evidence behind each rubric element

Rubric element	What the evidence establishes	Reference
Structure over scoreboards (signals 1–6)	Product quality is dominated by <i>maintainability</i> = modularity / low coupling / information hiding — a structural property, not a metric score	[R1] ISO/IEC 25010:2023
Anti-metric rule (§4.1): raw complexity is weak (bounds signal 11)	Cyclomatic / path / NPATH complexity correlate only weakly-to-moderately with real bugs; complexity is a <i>triage smell</i> , not a gate	[R2] Chen 2019
Anti-metric rule: Cognitive Complexity adds nothing	SonarSource's Cognitive Complexity gives no incremental predictive value over traditional measures	[R3] Lavazza et al. 2023
Anti-metric rule: SonarQube severities are weak / inverted	Over 33 Apache projects, "dirty" classes are no more fault-prone than clean ones; effects small, sometimes inverted	[R4] Lenarduzzi et al. 2020
Signal 7 — mutation as guidance (why coverage % hides shallow tests)	Mutation <i>score</i> is a poor linear proxy (suite-size artifact), but top-decile suites catch 8–46% more real faults — it exposes assertion-free tests that coverage % hides	[R5] Papadakis et al. 2018
Signal 5 / §3 — insecure code + overconfidence (mandates human review)	AI-assistant users wrote less-secure code yet were more confident it was secure	[R6] Perry et al. 2023
Signal 4 — hallucinated dependencies (slopsquatting)	LLM output frequently references nonexistent packages (~14.4% of distinct Python modules) → verify-before-add + hash-locked lockfile	[R7] Khanmohammadi et al. 2025
Signal 9 / §3 — silent duplication over reuse	2024 was the first year copy/pasted lines exceeded refactored ("moved") lines — the copy-instead-of-abstract signature (<i>vendor-sourced, correlational</i> — §7)	[R9] GitClear 2025
§3 — velocity ≠ delivered quality (the DORA context caveat)	AI adoption associated with lower delivery stability (2024, correlational, partly revised); experienced devs measured ~19% slower with early-2025 AI tooling	[R8] DORA 2024 · [R10] METR 2025

B.4 References

[R1] ISO/IEC 25010:2023. *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. International Organization for Standardization, 2023.

- [R2] Chen, C. (2019). *An Empirical Investigation of Correlation between Code Complexity and Bugs*. arXiv:1912.01142 [cs.SE]. <https://arxiv.org/abs/1912.01142>
- [R3] Lavazza, L., Abualkishik, A. Z., Liu, G., & Morasca, S. (2023). *An Empirical Evaluation of the "Cognitive Complexity" Measure as a Predictor of Code Understandability*. Journal of Systems and Software, 197, 111561. <https://doi.org/10.1016/j.jss.2022.111561>
- [R4] Lenarduzzi, V., Saarimäki, N., & Taibi, D. (2020). *Some SonarQube issues have a significant but small effect on faults and changes: A large-scale empirical study*. Journal of Systems and Software, 170, 110750. <https://arxiv.org/abs/1908.11590>
- [R5] Papadakis, M., Shin, D., Yoo, S., & Bae, D.-H. (2018). *Are Mutation Scores Correlated with Real Fault Detection? A Large Scale Empirical Study on the Relationship Between Mutants and Real Faults*. ICSE 2018. <https://doi.org/10.1145/3180155.3180183>
- [R6] Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). *Do Users Write More Insecure Code with AI Assistants?* ACM CCS 2023. arXiv:2211.03622. <https://arxiv.org/abs/2211.03622>
- [R7] Khanmohammadi, K., Roy, P., Khoury, R., Hamou-Lhadj, A., Konan, W. P., Da Re, A., & Rebelo Melo, N. (2025). *WildCode Revisited: A Comprehensive Empirical Study on the Security of LLM-Generated Code*. arXiv:2512.04259 [cs.CR]. <https://arxiv.org/abs/2512.04259> — the ~14.4% figure is the share of *distinct* Python modules; per-reference rate is far lower (§7).
- [R8] Google Cloud / DORA (2024). *Accelerate State of DevOps Report 2024*. <https://dora.dev/research/2024/> — AI adoption vs. delivery stability/throughput; associational, partly revised in 2025.
- [R9] GitClear (2025). *AI Copilot Code Quality 2025* ("AI-Generated Code Exerts Downward Pressures on Code Quality"). https://www.gitclear.com/ai_assistant_code_quality_2025_research — commercial vendor; proprietary "moved" reuse heuristic; AI attribution correlational (§7).
- [R10] METR (2025). *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*. arXiv:2507.09089. <https://arxiv.org/abs/2507.09089> — 16 experienced devs ~19% slower while believing they were faster.

B.5 What was refuted (the verification worked)

Five widely-circulated claims **failed** 3-vote adversarial verification and are **deliberately not** in the rubric — the filtering is part of the basis:

- "**~40% of GitHub Copilot programs contain security vulnerabilities.**" An over-simplified reading of the NYU "Asleep at the Keyboard" study (Pearce et al., IEEE S&P 2022, arXiv:2108.09293); the headline percentage did not survive scrutiny. (*The study is real; the "40%" framing is not a supported rubric claim.*)
- A "**10× surge in duplicate blocks**" (2022→2024) — GitClear figure, refuted 0–3.
- "**Copy/pasted lines rose 8.3%→12.3% (~48%)**" and "**moved operations fell 17% (2021→2023)**" — GitClear / secondary-blog percentages that did not reconcile, refuted 0–3.

Because the refuted claims came mostly from the *AI-slop-trend* angle (vendor telemetry, secondary blogs), the rubric leans hardest on the **peer-reviewed metric-validity studies** ([R2]–[R6]) and treats the trend data ([R8], [R9], [R10]) as caveated context (§7).

B.6 Caveats

The evidence caveats in §7 are part of this appendix's basis: the metric-invalidation findings are robust peer-reviewed results, but they are "this-metric-is-weak-*alone*" results (they argue against single-number gates, not against measurement); the AI-slop trend data is correlational / vendor-sourced / model-era-specific; and no source supplies a validated single-metric certification threshold.

Version history

Version	Date	Change
0.11	July 27, 2026	Added the liveness rule (new section 4.0) and built the control. v0.10 recorded that signal 7 had been scored  Built for two versions while its tool crashed before producing a mutant. That is a failure mode this rubric had no defence against: section 4.1 forbids over-trusting a <i>number</i> , but nothing forbade over-trusting a <i>green check that never ran</i> — and three defects across two of the five Tier 2 gates turned out to have that shape (two measuring nothing, one publishing a wrong derived number). Section 4.0 now requires every advisory gate to prove it measured something (units examined , never units found — a clean repo reports zero and must still pass) or to declare explicitly, with a reason, that it had nothing to measure; and any derived headline figure must be cross-checked against an independently produced measurement of the same quantity. Implemented as the <code>liveness</code> job in <code>quality-advisory.yml</code> — the only job there permitted to go red — with <code>tests/test_gate_liveness.py</code> replaying the historical incidents to prove the check catches them, and the good-news cases to prove it does not fire on them. The control was itself adversarially reviewed before merge, and the review found it carrying the same weakness it was built to catch, in three places — a dead coverage gate could pass by claiming "not applicable", an empty mutmut results file reported a flawless score, and the reconciliation sum was algebraically blind to the very count it claimed to protect. All three are fixed and regression-tested; rule 3 above was rewritten because of the third. No scoring change (A–stands); the gates were repaired in v0.10, this is the control that keeps them honest.
0.10	July 27, 2026	Restored to the repo, and corrected three claims that did not survive measurement. This file had been absent from the repository's entire git history despite being cited by <code>quality-advisory.yml</code> and <code>pyproject.toml</code> ; it is restored here from the maintained copy. Corrections, each measured rather than reasoned: (a) Signal 7 was scored  Built in 0.8 and 0.9 while producing nothing. <code>mutmut<3</code> resolved to 2.5.1, which crashes on Python 3.14 in its pony-ORM cache (<code>cannot pickle 'itertools.count'</code>) <i>before generating a single mutant</i> ; <code>\\ \\ true</code> made the job report success in 37s, so the gate looked green for two versions. Repaired on <code>mutmut==3.6.0</code> (+ <code>pytest-timeout</code> , and <code>source_paths</code> must be the package, not the one file, or the mutant copy cannot import <code>conftest</code>). Now genuinely measured: 461 mutants, 87 killed, 19 survived, 3 seconds — so the "Expensive / never per-PR" cost model in §5 was also wrong, and mutation now runs on PRs. (b) Signal 11's "85 functions over C901>10" is now 122 across 43 files , and the raw list was found unusable as a diff signal (every finding anchors on one <code>def</code> line), so a merge-base delta was added that reports only PR-caused changes. (c) Signal 8 now emits inline PR annotations rather than console-only output. The A– verdict stands, but note that (a) is exactly the failure mode this rubric exists to catch — an advisory gate that reports success while measuring nothing — and it was caught by re-verification, not by the gate itself.

Version	Date	Change
0.9	July 14, 2026	Restatused signal 10 (lint breadth) to  Built — all 11 signals now Built. The <code>extend-select = [B,C4,SIM,UP,I]</code> sweep shipped (#1047): B008 handled via <code>extend-immutable-calls</code> + a route-layer per-file ignore, 515 auto-fixed, 235 grandfathered with <code># noqa</code> , enforced by the required <code>ruff check</code> leg. Flipped the exec verdict, §5 gate table + callout, §6 map, and Appendix A.1 / A.2 (row 10 + Tier-2 roll-up) / A.3 (gaps list + "remaining gate" prose → rollout <i>record</i>). No scoring change (A– stands).
0.8	July 14, 2026	Restatused signals 7 (mutation) + 8 (diff-coverage) to  Built. Both shipped as advisory jobs in <code>quality-advisory.yml</code> (#1040) — mutation over a bounded module (mirror-nightly + <code>workflow_dispatch</code>), diff-coverage on the diff's changed lines (PR-only). Flipped every place that called them deferred: the exec verdict, §5 gate table + callout, §6 map, and Appendix A.1 / A.2 (rows 7–8 + the Tier-2 roll-up) / A.3 (gaps list + DEP-1 note) / A.4. Only signal 10 (lint breadth) remains designed-but-deferred → 10 of 11 signals now Built. No scoring change (A– stands).
0.7	July 13, 2026	Tiered the Appendix A scorecard to match §4 — split A.2 into a Tier 1 table (durable, signals 1–6, all Built) and a Tier 2 table (measurement, signals 7–11), each with a per-tier status roll-up. Presentational only; no scoring change.
0.6	July 13, 2026	Renumbered signals contiguously by tier (per owner): Tier 1 durable = 1–6 (published-artifact 11→6), Tier 2 measurement = 7–11 (mutation 6→7, coverage 7→8, clone 8→9, lint 9→10, complexity 10→11). Updated every cross-reference — §3, §4, §5, §6, exec summary, scope, and Appendix A.1 / A.2 (scorecard reordered) / A.3 / B.1 / B.2 / B.3. The <code>[R1]–[R10]</code> reference IDs are unchanged (they're citations, not signals).
0.5	July 13, 2026	Split the §4 matrix into two tiers for clarity: Tier 1 — durable, high-signal controls (signals 1–5 + 11) and Tier 2 — measurement / lower-signal layer (signals 6–10). Signal numbers kept stable (they are IDs referenced across Appendix A + B), so grouped by tier rather than renumbered. No change to content, evidence, or the scorecard.
0.4	July 13, 2026	Added Appendix B — the rubric matrix's methodology & cited references (B.1 how to read the matrix, B.2 derivation, B.3 per-element evidence map, B.4 full citations [R1–R10], B.5 the 5 refuted claims). Annotated the rubric with links to it: §2 + §3 evidence cells now carry <code>[Rn]</code> citation links, §4 has an evidence/citations pointer, and §8 points to the full bibliography. Fetched-verified author/title details for the academic references.
0.3	July 13, 2026	Built two gates + demoted DORA. Complexity (signal 10) and clone (signal 8) shipped as advisory jobs in <code>quality-advisory.yml</code> (#1028) — scorecard restatused to  Built (§5, A.2, A.3). Demoted delivery-stability (DORA) from a peer signal to a context caveat (it measures delivery outcomes, not the code artifact; weak/correlational evidence; owned by the AI companion) → back to 11 signals (published-artifact renumbered 12→11). Mutation (#6) + diff-coverage (#7) deferred to a local-venv session; ruff-breadth (#9) to a quiescent-worktree sweep.
0.2	July 13, 2026	Added signal 12 — published-artifact integrity (supply-chain- <i>out</i>) and the control-parity failure mode (§3), after the rubric's own application surfaced a private-doc leak in the published PyPI sdist (fixed #1020, verified clean at v0.3.0 — Appendix A.1 + A.2 row 12). Scorecard refreshed (12 signals).
0.1	July 13, 2026	Initial rubric. Evidence base from an adversarially-verified deep-research pass (20 confirmed / 5 refuted claims) + a read-only MEFOR code-quality audit. Establishes the anti-metric rule (§4.1), the 11-signal composite rubric (§4), the five new measurement gates with local-vs-CI placement (§5), and the MEFOR scorecard (Appendix A, verdict A–). Recorded as the third companion to the SDS + Secure AI-Assisted Development Standards.

© 2026 **MEFOR-ORG**, a non-profit · MessageFoundry is licensed under **AGPL-3.0-or-later** · Document generated July 2026 for MessageFoundry v0.3.2.

MessageFoundry and the MessageFoundry word mark are trademarks of MEFOR-ORG. This document describes an **Early Access** release; check pypi.org/project/messagefoundry for the current version. Verify specifics against your own deployment and the engine's reference documentation.